



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۱۱ آبان

فصل سوم: لایه *Transport* اسلاید ۳۲ تا ۵۴ [۱]

rdt 2.1

در این نوع *rdt*، برای بسته‌های *sequence number* داریم که از صفر شروع می‌شوند (۰، ۱، ۲، ۳، ...). در الگوریتم *stop and wait* می‌توانیم با *sequence number*‌های صفر و یک، ترتیب را رعایت کنیم و دیگر نیاز به *sequence number*‌های بیشتر نیست. فرستنده ۴ تا *state* دارد. یا منتظر *call* از بالا برای ارسال بسته صفر است، یا بعد از آن منتظر *ACK* گرفتن از ارسال بسته صفر است، بعد منتظر *call* از بالا برای ارسال بسته ۱ است و بعد منتظر *ACK* آن و دوباره این روند تکرار می‌شود.

در سمت گیرنده هم دو *state* داریم. یا منتظر دریافت صفر از پایین است یا منتظر دریافت بسته ۱. گیرنده می‌تواند در ازای دریافت هر بسته *ACK* یا *NAK* بفرستد.

یک سوال این بود اگر گیرنده در حالت انتظار برای بسته ۱ باشد اما بسته صفر از فرستنده دریافت کند، باز آن را *ACK* جواب می‌دهد اما *state* خود را تغییر نمیدهد. چرا؟ احتمالاً چون ممکن است *ACK* یا *NAK* به فرستنده نرسیده باشد و گیرنده از این که رسیده یا نه خبری نمیتواند داشته باشد،

rdt 2.2

در این نوع، گیرنده دیگر *NAK* نمی‌فرستد و به جای آن، *sequence number* آخرین بسته‌ای را که دریافت کرده را به همراه *ACK* می‌فرستد. یعنی اگر دو تا *ACK* با *sequence number* یکسان فرستاد، یعنی بسته آخر را دریافت نکرده.

rdt 3

در این نوع *rdt* علاوه بر *bit error* ممکن است *packet loss* هم داشته باشیم، چه پکت‌های فرستنده و چه پکت‌های *ACK*.

برای همین در سمت فرستنده، یک تایمر بعد از هر ارسال بسته فعال می‌شود، و اگر به هر دلیلی، بسته *ACK* را دریافت نکرد یا به دلیل ازدحام، *ACK* با تاخیر روبرو شد، بسته را باز ارسال می‌کند.

بهره‌وری *bandwidth* شبکه در این نوع *rdt*ها

از آنجایی که این روش‌ها *stop and wait* هستند، تنها یک بسته ارسال می‌شود و منتظر پاسخ میماند تا بسته بعد را ارسال کند. بنابراین کارایی و *utilization* خوبی ندارد. به همین دلیل، روش‌های *pipelining* معرفی شد که چندین بسته می‌تواند به صورت *in pipe* یا *in flight* منتقل شوند.

اضافه شدن امکان *pipeline*

با اضافه شدن این امکان، دیگر نمیتوان تنها با *sequence number*های صفر و یک کار کرد و باید با اعداد بیشتری کار کرد.

علاوه بر این باید در سمت مبدا و مقصد، بافر داشته باشیم.

اینگونه، بهره‌وری و *busy* بودن *sender* افزایش میابد.

دو روش *pipelined* عبارتند از *Go – Back – N* و *selective repeat*

روش *Go – Back – N*

در این روش *N* بسته را به صورت *in pipe* می‌فرستند و منتظر *ACK* تجمعی آن میماند. اگر تا *n* را به ترتیب و درست گرفته باشد، *ACK(n)* می‌فرستد یعنی *n* تا را گرفته بدون هیچ *packet loss*.

یک متد *nextSeqNum* هم داریم که برای شماره گذاری بسته‌ها برای *sequence number*شون استفاده می‌شود. (درواقع مثل *program counter* میماند، وقتی مقدار داده شد، به عدد بعدی اشاره میکند.) در پنجره فرستنده، این *nextSeqNum* از *base* تا *base + N* تغییر میکند. یعنی حداکثر *n* تا در هر پنجره.

در گیرنده هم یک *expectedSeqNum* داریم که درواقع آخرین بسته مورد انتظار به ترتیب گرفته شده بعدی است. مثلاً اگر منتظر بسته ۱۰۱ باشد (یعنی ۱۰۰ تا بسته اول را به ترتیب گرفته)، بسته‌های ۱۰۲ و ۱۰۳ را در صورت دریافت حذف میکند. اما یک *ACK* می‌فرستد که مطابق آن *expectedSeqNum* است تا فرستنده بگوید که این بسته را می‌خواهد. (نفهمیدم چرا استاد گفت همین *expectedSeqNum* را در *ACK* می‌فرستد و نه ی‌دونه کمترش رو)

اون *sequence number*های بعد از *packet loss* را در نظر نمی‌گیرد حتی اگر دریافت کرده باشد. و *ACK* آخرین بسته‌ای که تا آنجا کامل گرفته را برمیگرداند.

با دریافت *ACK* از گیرنده، *base* آپدیت می‌شود. و تا شماره آن بسته جلو میرود.

وقتی یک بسته با شماره n فرستاده (و بسته های بعدی را هم میفرستد) اما همش دارد ACK با شماره $n-1$ میگیرد، تایمر آن بسته n که فعال شده و تمام میشود و از اون n به بعد رو دوباره میفرستد. (در واقع ما $partial ACK$ داشتیم اینجا)

هر وقت $base$ به $nextSeqNum$ برسد، یعنی کل پنجره را ارسال کرده است.

روش *selective repeat*

توی این روش، گیرنده تمام بسته هایی رو که به درستی دریافت کرده، ACK شان را میفرستد، صرف نظر از اینکه همه قبلی ها را ACK کرده یا نه. فرستنده فقط اون بسته هایی که ACK شان نیامده و تایمرش تمام شده، باز ارسال میکند.

مطمئن نیستیم ولی فکر کنم بسته های خارج از ترتیب، بافر می شوند. (ولی ACK آنها ارسال می شود به هر حال)

مشکل روش *selective repeat*

مشکل این است که بازه $sequence number$ کمتر از سایز پنجره است. بعد طبق مثالی که در اسلاید زده، در حالت اول بسته شماره ۵ که $sequence number = 0$ دارد را ارسال میکند (چون $0, 1, 2, 3, 0, 1, 2, 3, 0, \dots$ است). در حالت دوم، چون ACK از گیرنده نگرفته، بسته اول با $sequence number = 0$ ارسال می کند. در هر دو حالت گیرنده بسته با $sequence number = 0$ می بیند. پس به مشکل می خورد. ممکن است در حالت دوم، فکر کند بسته پنجم را دریافت کرده است.

مراجع

[1] “ch3-v8-slide-32-to-54,” <https://drive.google.com/drive/folders/>, 2022.